

BUBBLE BLASTER

Figuring out the distance between points

In this game, and lots of others, it is useful to know the distance between two objects. Here's how to use a well-known mathematical formula to have the computer work it out.

11 This function calculates the distance between two objects. Add this bit of code directly after the code you wrote in step 9.

```
from math import sqrt
def distance(id1, id2):
    x1, y1 = get_coords(id1)
    x2, y2 = get_coords(id2)
    return sqrt((x2 - x1)**2 + (y2 - y1)**2)
```

Loads the "sqrt" function from the Math library

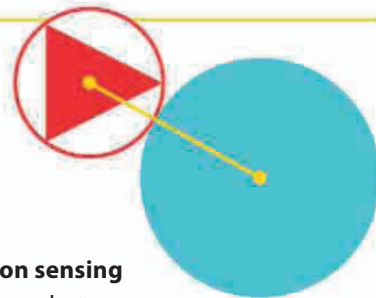
Gets the position of the first object

Gets the position of the second object

Gives back the distance between them

Pop the bubbles

The player scores points by popping bubbles. Big bubbles and fast bubbles are worth more points. The next section of code works out when each bubble is popped by using its radius (the distance from the center to the edge).



► Collision sensing

If the distance between the center of the sub and the center of a bubble is less than their radiuses added together, they have collided.

12 When the submarine and a bubble crash into each other, the program needs to pop the bubble and update the score. This bit of code should come directly after the code in step 11.

```
def collision():
    points = 0
    for bub in range(len(bub_id)-1, -1, -1):
        if distance(ship_id2, bub_id[bub]) < (SHIP_R + bub_r[bub]):
            points += (bub_r[bub] + bub_speed[bub])
            del_bubble(bub)
    return points
```

This variable keeps track of points scored

This loop goes through the entire list of bubbles (it goes backwards to avoid errors when deleting bubbles)

Checks for collisions between the sub and any bubbles

Calculates the number of points this bubble is worth and adds it to "points"

Deletes the bubble

Gives back the number of points